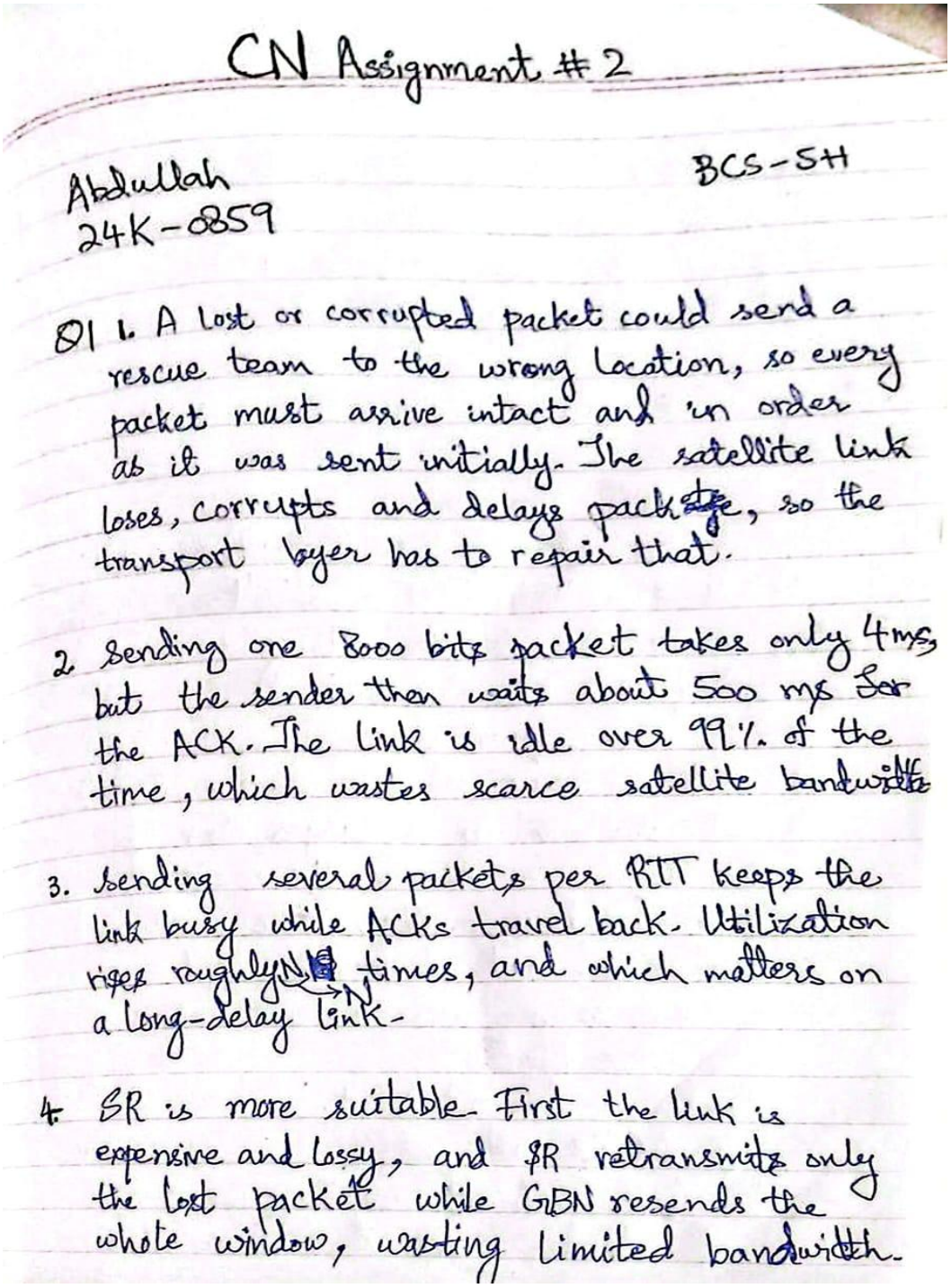
CN Assignment#2

Abdullah

24K-0859

Tasks 1 & 5:



CN Assignment #2

Abdullah
24K-0859

BCS-5H

Q1 1. A lost or corrupted packet could send a rescue team to the wrong location, so every packet must arrive intact and in order as it was sent initially. The satellite link loses, corrupts and delays packets, so the transport layer has to repair that.

2. Sending one 8000 bits packet takes only 4ms, but the sender then waits about 500 ms for the ACK. The link is idle over 99% of the time, which wastes scarce satellite bandwidth.

3. Sending several packets per RTT keeps the link busy while ACKs travel back. Utilization rises roughly N times, and which matters on a long-delay link.

4. SR is more suitable. First the link is expensive and lossy, and SR retransmits only the lost packet while GBN resends the whole window, wasting limited bandwidth.

Secondly, with a 500 ms RTT, each GBN retransmission burst costs a full extra RTT of delay, while SR's reciever buffering avoids discarding good packets.

Q5 1. Utilization $\frac{L}{R} = \frac{8000}{2{,}000{,}000} = 4\text{ms}$

RTT = 500 ms

- Stop and Wait: $U = 4/(500+4) \approx 0.0079\ (0.79\%)$
- N = 4: $U = (4\times4)/504 = 16/504 \approx 0.0317\ (3.17\%)$

Pipeling improves utilization about 4 times; 4 packets in flight per RTT.

2. Sequence Numbers (k=3)

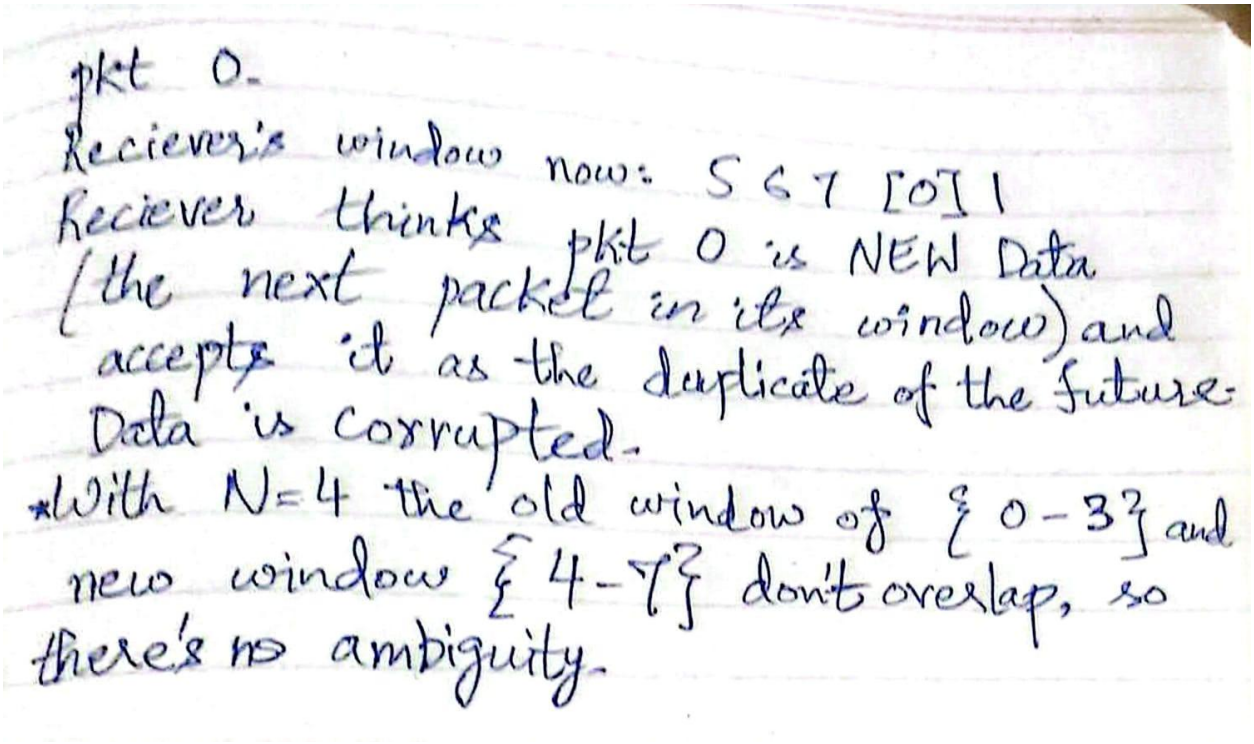
a) $2^3 = 8$ sequence (0-7)

b) SR maximum safe window $= 2^{k-1} = 4$

c) If the window is larger than half the sequence space, the reciever's new window overlaps the senders old one, and it can't tell a retransmission from a new data.

Example N=5:

Sender sends 0,1,2,3,4. Reciever gets all, ACKs all and then slides window to [5,6,7,0,1]. All 5 ACKs are lost. Sender times out retransmits



Task 2:

Packet trace (N = 4, packet 2 lost), I am Assuming: ACK0 and ACK1 return before the timeout, so the window slides and packets 4 and 5 are also sent. The receiver therefore sees out-of-order packets 3, 4 and 5.

(A) Go-Back-N

Event	Sender	Packet	Receiver	ACK	Window
1	Send pkt 0	0			[0–3]
2	Send pkt 1	1			[0–3]
3	Send pkt 2	2	**Lost**		[0–3]
4	Send pkt 3 (window full)	3			[0–3]
5		0	Delivered, expectedseqnum = 1	ACK0	
6	Get ACK0, base = 1, send pkt 4	4			[1–4]
7		1	Delivered, expectedseqnum = 2	ACK1	

8	Get ACK1, base = 2, send pkt 5	5			[2–5]
9		3	Out of order, **discarded**	ACK1 (dup)	
10		4	Discarded	ACK1 (dup)	
11		5	Discarded	ACK1 (dup)	
12	Duplicate ACKs ignored, base stays 2				[2–5]
13	**Timeout for pkt 2**, resend 2, 3, 4, 5	2,3,4,5			[2–5]
14		2,3,4,5	Each in order, delivered, expectedseqnum = 6	ACK2, ACK3, ACK4, ACK5	
15	Get ACK2 → base 3, send 6	6			[3–6]
16	Get ACK3 → base 4, send 7	7			[4–7]
17	Get ACK4 → base 5, send 8	8			[5–8]
18	Get ACK5 → base 6, send 9	9			[6–9]
19		6,7,8,9	Delivered, expectedseqnum = 10	ACK6–ACK9	
20	Get ACK9, base = 10, done				empty

(B) Selective Repeat:

Event	**Sender**	**Packet**	**Receiver**	**ACK**	**Sender / receiver window**
1–4	Send 0, 1, 2, 3	0,1,2,3	Pkt 2 **lost**		S [0–3]
5		0	Delivered, rcv_base = 1	ACK0	R [1–4]
6	Get ACK0, send_base = 1, send 4	4			S [1–4]
7		1	Delivered, rcv_base = 2	ACK1	R [2–5]
8	Get ACK1, send_base = 2, send 5	5			S [2–5]
9		3	**Buffered** (rcv_base still 2)	ACK3	R [2–5]
10		4	Buffered	ACK4	
11		5	Buffered	ACK5	
12	Marks 3, 4, 5 as ACKed, send_base stays 2				S [2–5]
13	**Timeout for pkt 2**, resend **only 2**	2			
14		2	Delivers 2, 3, 4, 5 in order, rcv_base = 6	ACK2	R [6–9]
15	Get ACK2, send_base jumps to 6, send 6, 7, 8, 9	6,7,8,9			S [6–9]
16		6,7,8,9	Delivered in order, rcv_base = 10	ACK6–ACK9	
17	send_base = 10, done				empty

Final Answer:

GBN retransmits 4 packets (2, 3, 4, 5), so 14 transmissions in total. SR retransmits 1 packet (2), so 11 in total. They differ because the GBN receiver threw away packets 3, 4 and 5 even though they arrived correctly, so the sender must resend them. The SR receiver buffered them, so only the truly missing packet needs resending. If only packets 3 and 4 had reached the receiver before the timeout, GBN would retransmit 2, 3 and 4 (3 packets) and SR would still retransmit only packet 2 (1 packet).

Task 3: Implementation

I wrote everything in one file, reliable_transfer.py. It runs as a simulation where events wait in a queue ordered by time, so the same seed always gives the same run. For every packet and ACK, the channel randomly decides whether it is lost, corrupted or delayed (up to 150 ms extra). Corruption flips a bit, and a 16-bit checksum lets the receiver spot it and drop the packet. Sequence numbers wrap modulo 2^k, and every send, receive, ACK, loss, timeout and retransmission is logged.

GBN. The sender tracks base and nextseqnum and uses one timer for the oldest unACKed packet. ACKs are cumulative, and a timeout resends everything from base to nextseqnum - 1. The receiver accepts only the packet equal to expectedseqnum, discards the rest and repeats the ACK for the last in-order packet.

SR. Each packet has its own timer and acked flag, and a timeout resends only that packet. The receiver keeps its own window from rcv_base, buffers out-of-order packets, ACKs each one individually and delivers data in order. It also re-ACKs packets just below its window, so a lost ACK doesn’t leave the sender stuck.

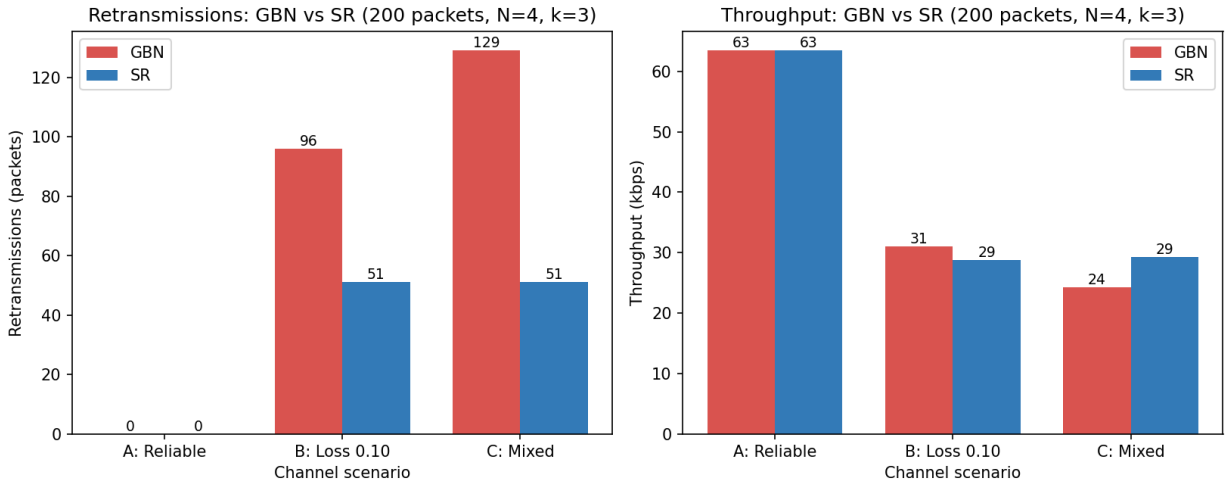
A problem I faced: My first timeout of 600 ms was shorter than the worst-case round trip in my channel (about 800 ms: 500 ms plus up to 150 ms extra each way). Early timeouts created duplicate copies, and sometimes an old copy arrived after the sequence numbers had wrapped, so SR accepted it as new data. A 900 ms timeout fixed this, and scenario C then delivered all 200 packets correctly across the 300 seeds I tried.

I Assumed that ACKs are tiny and take no transmission time, and the timeout must stay above the worst-case RTT.

Task 4: Experiments and Results

Settings: 200 packets, N = 4, k = 3, seed = 859. A is a reliable channel, B has loss 0.10, and C has loss 0.05, corruption 0.05 and delay 0.10. Every run delivered all 200 packets correctly and in order.

Scenario	Protocol	Transmitted	Retransmissions	Delivered	Time (s)	Throughput (kbps)
A: Reliable	GBN	200	0	200	25.2	63.5
A: Reliable	SR	200	0	200	25.2	63.5
B: Loss 0.10	GBN	296	96	200	51.5	31.1
B: Loss 0.10	SR	251	51	200	55.6	28.8
C: Mixed	GBN	329	129	200	65.9	24.3
C: Mixed	SR	251	51	200	54.7	29.3



Both protocols perform the same on a reliable channel, since nothing needs recovering. With loss or corruption, GBN resends the whole window after a timeout, including good packets the receiver discarded, so it retransmits far more (96 vs 51 in B, 129 vs 51 in C). In scenario C SR also finished faster (54.7 s vs 65.9 s). In scenario B GBN had slightly higher throughput on this seed (31.1 vs 28.8 kbps), so the throughput gap is small and depends on the seed, and the retransmission counts are the more reliable comparison.

Demo Video:

https://drive.google.com/file/d/1wcIuNaYJ5lBUs48r9ZHdRMK7FiuT47pT/view?usp=drive_link

Conclusion:

Both protocols deliver every packet correctly and in order, and both cope with loss, corruption and delay. GBN is simpler but wastes bandwidth by resending good packets. SR resends only what is lost, which suits an expensive, lossy satellite link better. Two practical lessons from this work were that the timeout must stay above the worst-case RTT and that the window must stay within half of the sequence-number space.